CometChat React

How the chat app was built, how every file connects, and what comes next

Project folder	C:\Users\kshit\howl\commetchat
Stack	React 19, Vite 8, CometChat JavaScript SDK 4.2.1
Status	Frontend built and compiling. No custom backend yet (see section 1).
Date	15 October 2026

Contents

1. Frontend and backend: what is built
2. The big picture (architecture diagram)
3. How it was built, step by step
4. Folder map: every file and what it connects to
5. How the files connect (dependency diagram)
6. What happens at runtime
7. Roadmap: what comes next
8. How to run it

1. Frontend and backend: what is built

The short answer: **the frontend is built. A backend of your own is not built yet.**

This works because CometChat is a hosted chat service. Their servers already do the jobs a chat backend normally does: storing users and messages, delivering messages in real time, tracking who is online, and sending read receipts. Our React app connects straight to their servers using their SDK, so for development no server of our own is needed.

Part	Who runs it	Status	What it does
React frontend	Us (this folder)	**Built**	Login screen, conversation list, chat window, typing indicators, read ticks, online status.
Chat backend	CometChat (hosted)	**Ready to use**	Stores users, groups and messages. Pushes live events over a WebSocket.
Token server	Us (to build)	**Not built**	A small server that creates CometChat users and gives each user a login token. Needed before going to production.

Why the token server matters
Right now the app logs in with the **Auth Key** stored in the .env file. Anyone who has that key can log in as any user, which is fine for development but not safe for real users. In production, your own server checks who the user is, then asks CometChat for a login token for that one user. The app logs in with that token and never sees a master key. This is Phase 2 of the roadmap.

2. The big picture

The app is built in layers. The UI never calls CometChat directly. Components use hooks, hooks use the service file, and only the service file sets up and calls the SDK. The SDK then talks to CometChat Cloud in two ways:

- **HTTPS requests** for things the user starts: logging in, loading old messages, sending a message.
- **A WebSocket**, a connection that stays open so CometChat can push events to the app the moment they happen: a new message, someone typing, a message being read, someone coming online.

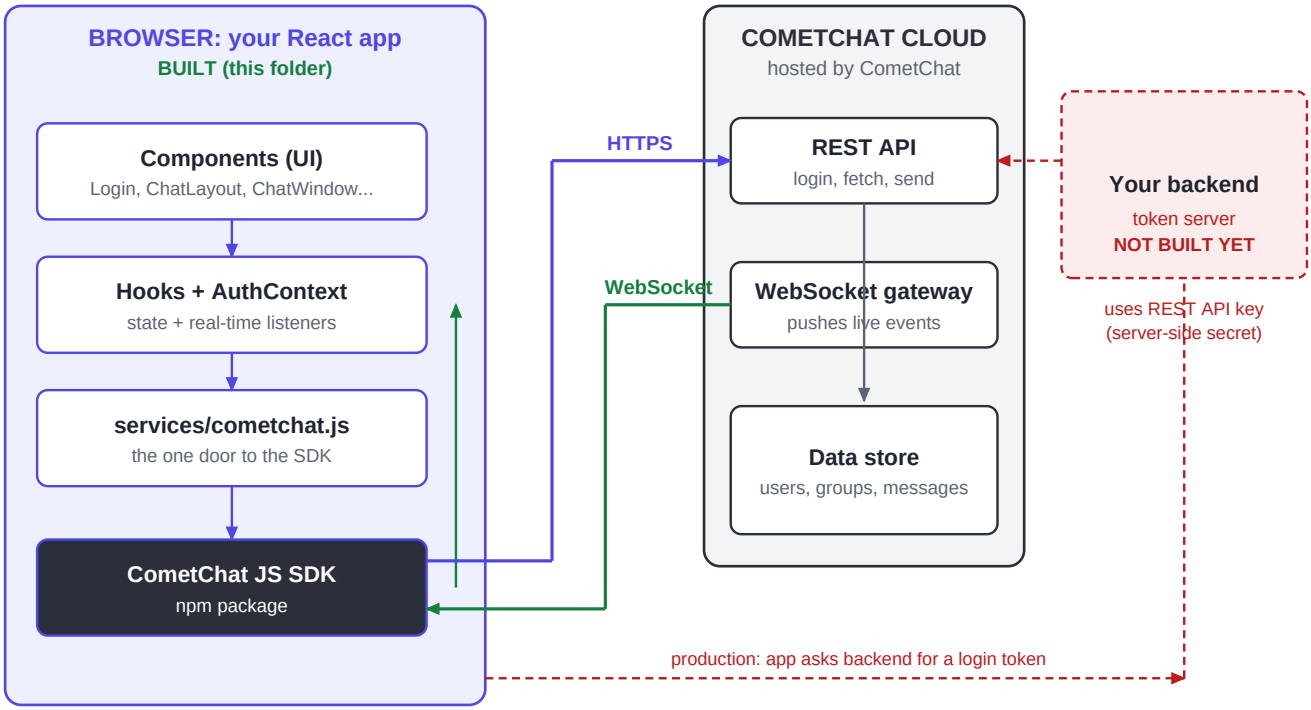

Figure 1. Blue arrows are requests the app makes. Green arrows are events pushed back up to the hooks. Red dashed parts are the production backend that is not built yet.

How a message travels

Here is what happens when Alice sends Bob a message. Both are running this same React app in their own browsers.

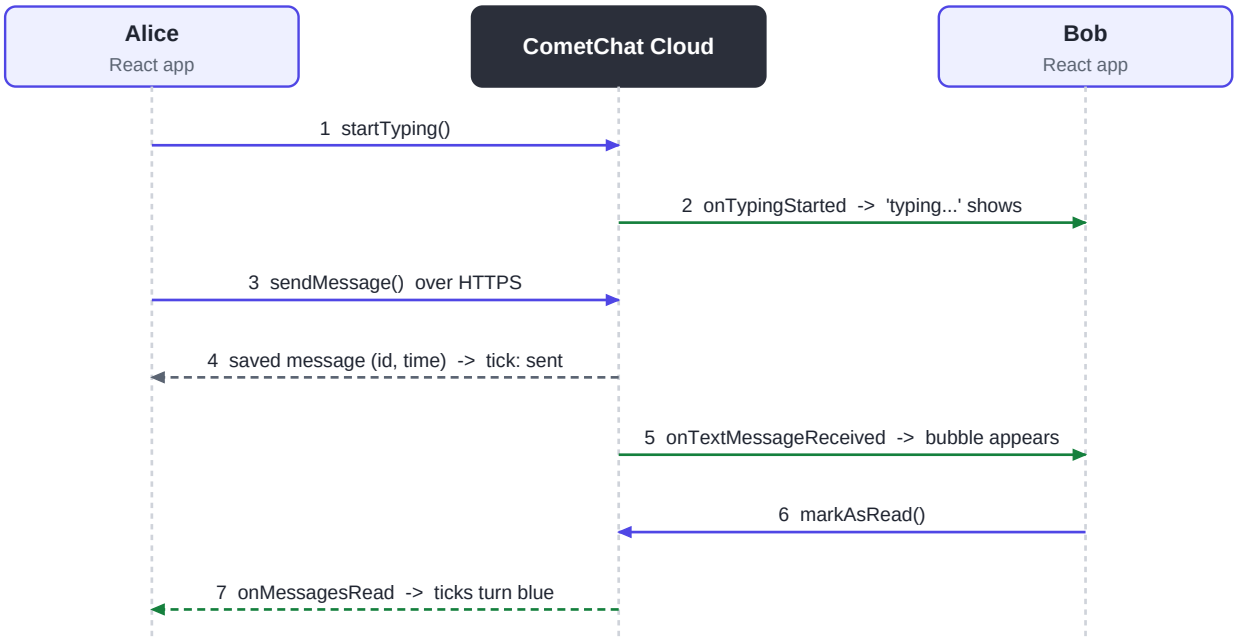

Figure 2. Solid arrows are actions. Dashed arrows are replies to the sender.

3. How it was built, step by step

The app was built from the bottom layer up. Each step depends only on the ones before it, so every layer could be written against something that already existed.

#	Step	Files	Why this step
1	Create the project	`package.json, vite.config.js, index.html, .gitignore`	Vite gives a fast dev server and production build for React.
2	Install packages	`node_modules (react, react-dom, @cometchat/chat-sdk-javascript, vite)`	The SDK provides login, messaging and real-time events.
3	Configuration	`.env.example`	App ID, Region and Auth Key come from the CometChat dashboard. Keeping them in .env means they are never written into the code.
4	Service layer	`src/services/cometchat.js`	Puts SDK setup, login, fetching and sending in one file. It also turns SDK objects into simple "chat target" objects the rest of the app can use.
5	Login state	`src/context/AuthContext.jsx`	Starts the SDK once, restores the session after a page refresh, and shares the logged-in user with every screen.
6	Real-time hooks	`src/hooks/useConversations.js, useMessages.js`	Hold the chat state and register the SDK listeners that receive live events. Each listener is removed when its screen closes, so messages are never duplicated.
7	UI components	`src/components/*.jsx, src/App.jsx, src/main.jsx`	Login screen, sidebar with Chats and Users tabs, chat window, message bubbles and the message box.
8	Styling	`src/index.css`	Light and dark themes. On phones the sidebar and chat each fill the screen.
9	Check the build	`npm run build`	Confirmed that the whole app compiles with no errors.
10	Write docs	`README.md, docs/ARCHITECTURE.md, this PDF`	Setup steps, diagrams and this guide.

> **Not tested yet**
> The app compiles, but it has not been run against a real CometChat app because that needs your App ID and Auth Key. Section 8 explains how to add them and test with two browser windows.

4. Folder map

Every file in the project, what it does, and which other files it connects to. **node_modules** (installed packages) and **dist** (build output) are generated, so they are left out.

```
commetchat/
|-- index.html              page shell, loads src/main.jsx
|-- package.json            packages + scripts (dev, build, preview)
|-- vite.config.js          Vite + React plugin
|-- .env.example            template for your CometChat keys
|-- README.md               setup steps
|-- docs/
|   |-- ARCHITECTURE.md     diagrams (Mermaid)
|   `-- CometChat-Project-Guide.pdf   this document
`-- src/
    |-- main.jsx            starts React
    |-- App.jsx             chooses which screen to show
    |-- index.css           all styles
    |-- services/cometchat.js   the one door to the SDK
    |-- context/AuthContext.jsx login state
    |-- hooks/              useConversations.js, useMessages.js
    |-- components/         8 UI components
    `-- utils/format.js     time formatting
```

Root files

File	What it does	Connects to
`index.html`	The single HTML page. Contains an empty #root div.	Loads src/main.jsx
`package.json`	Lists packages and the dev / build / preview commands.	Read by npm and Vite
`vite.config.js`	Turns on the React plugin for Vite.	Used by npm run dev / build
`.env.example`	Template for VITE_COMETCHAT_APP_ID, _REGION, _AUTH_KEY. Copy it to .env.	Read by services/cometchat.js

Source files (src/)

File	What it does	Uses (imports)
`main.jsx`	Starts React and wraps the app in AuthProvider.	AuthContext, App, index.css
`App.jsx`	Shows one of: setup message, loading, Login, or ChatLayout, based on login state.	AuthContext, Login, ChatLayout
`services/cometchat.js`	Starts the SDK, logs in/out, fetches conversations, users and messages, sends messages and typing events. Also has helpers such as targetKey and previewText.	CometChat SDK, .env values
`context/AuthContext.jsx`	Runs init once, restores the logged-in user, provides signIn and signOut to every screen.	services/cometchat.js
`hooks/useConversations.js`	Sidebar data: conversation list, unread counts, moves a chat to the top on a new message, online/offline status.	services/cometchat.js
`hooks/useMessages.js`	Open-chat data: history (30 at a time), live messages, delivered/read ticks, who is typing, send().	services/cometchat.js
`components/Login.jsx`	UID input plus buttons for the 5 sample users.	AuthContext

File	What it does	Uses (imports)
components/ChatLayout.jsx	Main screen: sidebar with Chats/Users tabs and the chat pane. Tracks which chat is open.	useConversations, ConversationList, UserList, ChatWindow, Avatar, AuthContext
components/ConversationList.jsx	Chats tab: name, last message, time, unread badge.	Avatar, services, utils/format
components/UserList.jsx	Users tab: everyone you can start a chat with.	Avatar, services
components/ChatWindow.jsx	Header (name, online, typing) plus message list plus input.	useMessages, MessageList, MessageComposer, Avatar
components/MessageList.jsx	Message bubbles, ticks, "Load older" button, keeps scroll in the right place.	services, utils/format
components/MessageComposer.jsx	Text box. Enter sends, Shift+Enter adds a line. Sends typing start/stop.	services
components/Avatar.jsx	Profile picture or initials, with a green online dot.	(none)
utils/format.js	Turns CometChat timestamps (seconds) into readable times.	(none)
index.css	All styles, light/dark theme, mobile layout.	(none)

5. How the files connect

Arrows point from a file to the file it renders or uses. Read it top to bottom: main.jsx starts everything, the screens sit in the middle, and every path to CometChat goes through the amber service layer.

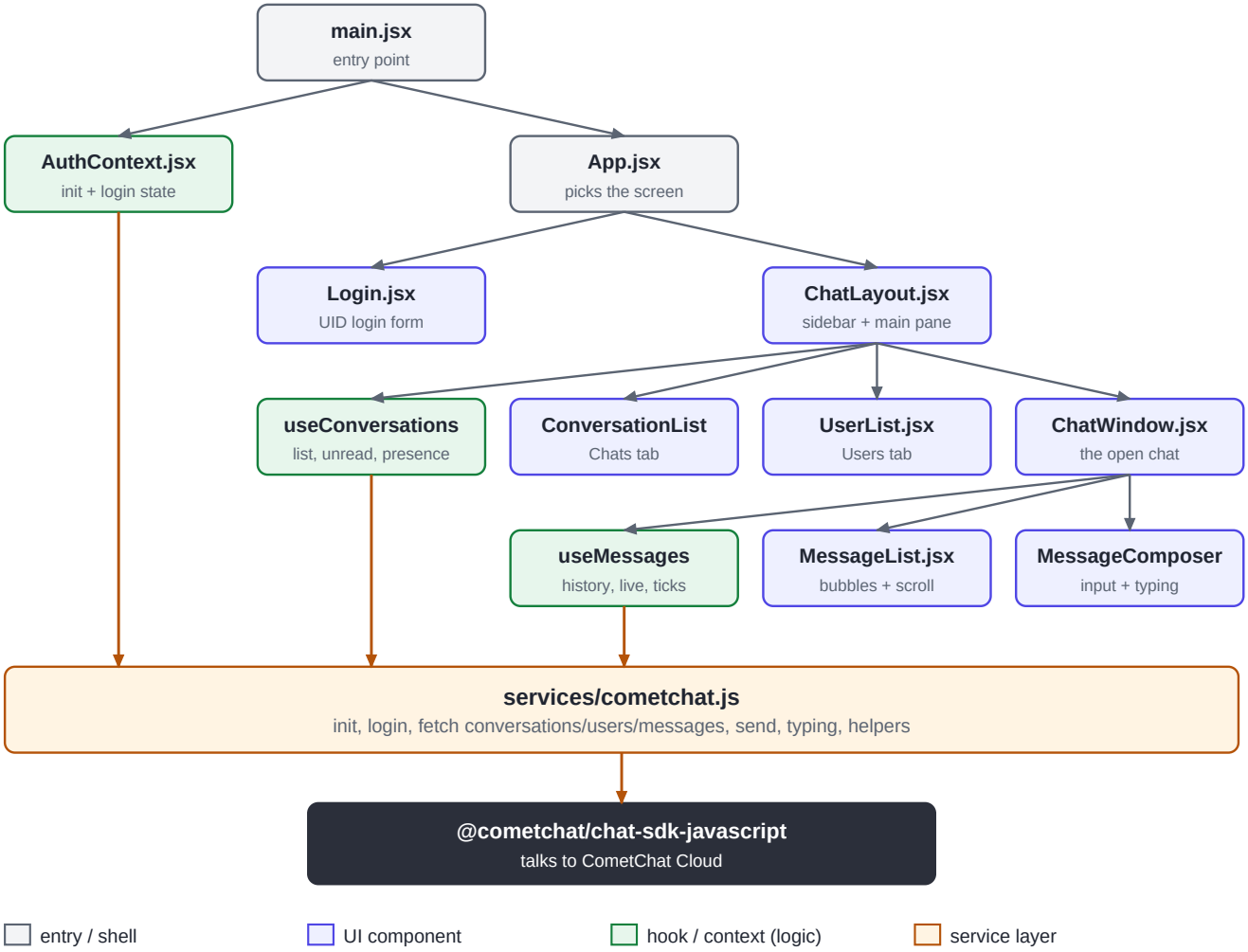

Figure 3. File dependency map. Avatar.jsx and utils/format.js are small shared helpers used by several components and are left out to keep the picture readable.

The three rules that keep it tidy

- **Only one file sets up and calls the SDK.** If CometChat changes its API, only services/cometchat.js needs to change. (The two hooks also register their live-event listeners on the SDK directly.)
- **Hooks hold state, components draw it.** Components such as ChatWindow get their data from a hook and only decide how it looks.
- **A chat is identified by a simple key.** "user:cometchat-uid-2" or "group:my-group". Every incoming message is matched to its chat with that key.

6. What happens at runtime

When the app opens

#	What happens	Where
1	index.html loads main.jsx, which renders AuthProvider and App.	main.jsx
2	If .env is missing, a setup message is shown instead.	App.jsx
3	CometChat.init() connects to your CometChat app using the App ID and Region.	AuthContext, services
4	getLoggedinUser() checks for an existing session, so a refresh keeps you logged in.	AuthContext
5	No session: the Login screen calls CometChat.login(uid, authKey).	Login.jsx
6	Logged in: ChatLayout loads the conversation list and starts listening for messages and online/offline events.	useConversations

When you open a chat

#	What happens	Where
1	The last 30 messages are fetched. "Load older" fetches 30 more each time.	useMessages
2	The newest message from the other person is marked as read, which turns their ticks blue.	useMessages
3	A listener for this chat starts. It adds new messages, updates ticks, and shows "typing...".	useMessages
4	The unread badge for this chat is cleared.	ChatLayout
5	Closing or switching the chat removes the listener.	useMessages cleanup

When you send a message

#	What happens	Where
1	Typing starts the typing indicator. It stops after 2 seconds without typing.	MessageComposer
2	Enter calls sendMessage(). CometChat saves it and returns it with an ID.	useMessages, services
3	The message is added to the chat and moved to the top of the sidebar.	useMessages, useConversations
4	If sending fails, the text goes back into the box with an error, so nothing is lost.	MessageComposer

7. Roadmap: what comes next

Phase	What to build	Details
1. React frontend **Done**	Login, conversations, 1:1 and group chat, typing, receipts, presence, history paging.	Everything in this folder today.
2. Token backend **Next**	A small Node.js (Express) server.	Endpoints: sign up (creates the CometChat user through the REST API) and get token (returns a login token for the signed-in user). The app switches to CometChat.login(authToken) and the Auth Key is removed from the frontend.
3. Media and groups	Images and files, emoji reactions, edit and delete, create and join groups.	Uses MediaMessage, the group APIs, and new UI in the composer and sidebar.
4. Notifications	Push notifications and unread count in the browser tab.	CometChat push extension, plus a service worker in the app.
5. Deploy	Host the frontend (e.g. Vercel or Netlify) and the token server.	Production .env values, HTTPS, and limiting which website addresses can use your CometChat app.

8. How to run it

- Create a free app at app.cometchat.com and open **API & Auth Keys**.
- Copy `.env.example` to `.env` and fill in the App ID, Region and Auth Key.
- In the project folder, run `npm install`, then `npm run dev`.
- Open the address Vite prints (usually http://localhost:5173) in a normal window and a private window.
- Log in as `cometchat-uid-1` in one and `cometchat-uid-2` in the other, then send messages back and forth.